# Toleranța la defectări

**Cuprins**
Concepte generale
Multicast fiabil si atomic
Recuperarea

**Terminologie**
O *specificatie* descrie comportarea ideala a unui *sistem* la *interfetele* sale. **Esecul** (Failure) este o deviere de la specificatia interfetei. Ea este provocata de o **eroare** (error), o stare interna (invizibila la interfata), care poate conduce la esec. **Defectul** (fault) reprezinta **cauza** unei erori. Dependenta cauzala intre aceste concepte este:

Defect ➔ Eroare ➔ Esec

**Exemplu**: praful determina reducerea turatiei unui ventilator care va produce mai putin aer si va determina supraincalzirea sursei de alimentare; o componenta arde si sursa se opreste. Gasim aici doua sisteme.
In **sistemul de ventilatie**, praful este cauza, reducerea turatiei reprezinta eroarea, iar diminuarea cantitatii de aer este esecul.
In al doilea sistem, **sursa de alimentare**, esecul ventilatiei reprezinta cauza, supraincalzirea sursei este eroarea, iar oprirea functionarii este esecul.

**Sisteme de incredere**
**Un sistem de incredere (dependable system) respecta specificatia chiar in prezenta defectarilor**. Principalele atribute ale sistemelor distribuite de incredere sunt definite in continuare. Desigur, unele atribute se pot asocia, intr-o forma sau alta, si altor categorii de sisteme.
Disponibilitatea (Availability) – reprezinta proprietatea unui sistem de a putea fi folosit imediat. Cu alte cuvinte, este probabilitatea ca un sistem sa lucreze corect la un moment dat.
Fiabilitatea (Reliability) – arata cat timp un sistem distribuit poate functiona continuu fara esec.
Siguranta (Safety) – este proprietatea ca daca un sistem functioneaza temporar incorect acest lucru sa nu aiba efecte catastrofice.
Mentenabilitatea (Maintainability) – arata cat este de usoara repararea unui sistem defect.

## Tipuri de defecte si esecuri

Tratarea defectelor depinde de tipul lor. **Defectele** pot fi clasificate in urmatoarele categorii:
- defecte tranzitorii – sunt cele care se produc o data dupa care dispar;
- intermitente – sunt defecte care apar repetat dar aleator; sunt foarte greu de diagnosticat;
- permanente – se manifesta continuu pana cand defectul este indepartat.

Si cunoasterea esecurilor este importanta pentru gasirea solutiilor de tratare a defectelor corespunzatoare. O posibila schema de clasificare este data in tabelul urmator, in care diversele categorii de esecuri sunt exemplificate prin descrierea comportamentului unui server care esueaza.

| Tipuri de esecuri | Descriere comportare server |
|---|---|
| **Cadere** (Crash) | Un server se opreste dintr-o data, dupa ce a functionat corect pana la cadere |
| **Omisiune**<br>*receptie*<br>*transmisie* | Nu raspunde cererilor<br>Nu primeste mesaje (nu din cauza conexiunii intre client si server)<br>Nu trimite mesaje (desi serverul a prelucrat cererea clientului) |
| **Timing** | Raspunde dupa trecerea timpului specificat |
| **Raspuns**<br>*valoare*<br>*tranzitie* | Raspunde incorect<br>Valoarea din raspuns este gresita<br>Deviaza de la fluxul de control corect facand actiuni necorespunzatoare |
| **Arbitrar** (Bizantin) | Produce raspunsuri arbitrare la momente arbitrare |

## Tehnici de tratare a defectelor

**Prevenire**
Se realizeaza prin metode de ingineria programelor cum ar fi: proiectare si verificare formala, controlul calitatii, injectare de defecte si testare.
**Detectie**
De regula, procesele ne-defecte detecteaza procesele defecte, atunci cand la trimiterea unor mesaje de interogare nu primesc raspuns. Pentru siguranta detectiei, mai multe procese isi coreleaza rezultatele interogarilor.
**Recuperare**
Se face prin readucerea componentei defecte intr-o stare corecta de functionare.
**Prezicere**
Reprezinta identificarea unor secvente de stari despre care se stie ca duc la defectare.
**Tolerare**
Furnizarea serviciului chiar daca apar defecte, eventual cu performante mai slabe. Se realizeaza prin **mascarea** defectelor.

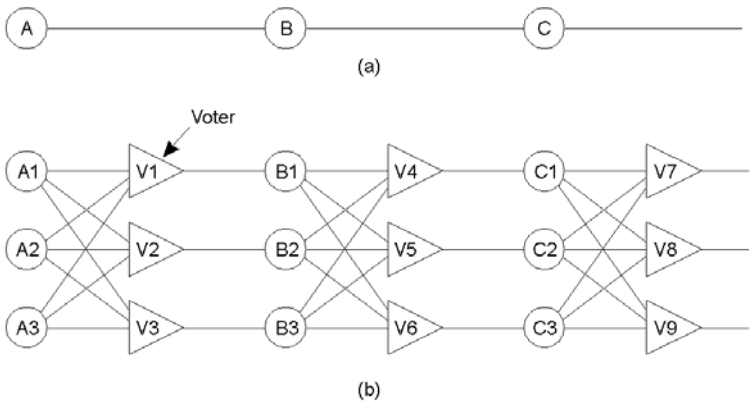
## Mascarea esecurilor

Metoda uzuala de toleranta la defectari este replicarea, care a facut subiectul altui capitol al cursului. O problema importanta este stabilirea gradului de replicare, in functie de tipul de esec si de gradul de toleranta dorita. Un sistem **k-fault tolerant** poate sa mascheze k procese defecte. Numarul total de replici, **n** se calculeaza cu formulele:
$n = k+1$ pentru esecuri de tip crash (este suficient ca o replica sa functioneze corect si sa ofere un rezultat daca replicile defecte nu ofera nici un rezultat)
$n = 3k+1$ pentru esecuri bizantine (solutie Lamport). Situatia este complicata de faptul ca replicile defecte ofera rezultate, dar care sunt incorecte. Este necesar ca numarul de replici corecte sa fie strict mai mare ca de doua ori numarul de replici defecte (justificarea este prezentata la discutia algoritmului generalilor bizantini (curs APD).

**Redundanta modulara tripla** reprezinta o forma particulara de redundanta si anume redundanta fizica. In exemplul din figura, echipamentele A, B si C sunt legate in serie (figura a). Pentru a

asigura toleranta, fiecare echipament este replicat de trei ori. Dupa fiecare grup de trei replici se afla trei echipamente de votare. Fiecare echipament de votare primeste semnale de la cele trei replici si decide valoarea iesirii pe principiul majoritatii (daca doua replici dau aceeasi intrare aceasta este selectata pentru iesire). Schema permite defectarea cate uneia din replicile grupurilor A, B si C fara ca rezultatele la cele trei iesiri ale ansamblului de echipamente sa fie afectate. In plus, replicarea echipamentelor de votare permite obtinerea unui rezultat corect chiar daca apare un defect la fiecare din cele trei grupuri de echipamente de votare.

**Exercitiu**: elaborati justificarea pentru afirmatiile anterioare.




## Comunicare de grup fiabila

Intr-o transmitere multicast fiabila, un mesaj trimis unui grup de procese trebuie livrat fiecarui membru al grupului. Esecurile se pot datora pierderii de mesaje sau defectarii proceselor din grup. Presupunem deocamdata ca grupul este stabil (procesele nu "crapa" si nu se alatura grupului in timpul comunicarii) si consideram doar defecte de comunicare.

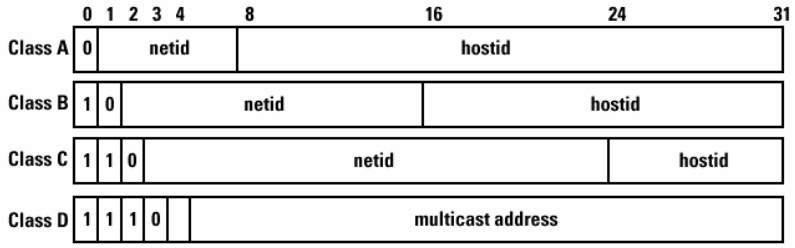
Transmiterea multicast fiabila este utilizata in mai multe aplicatii si protocoale, dintre care mentionam urmatoarele:
– Tranzactii fiabile si ordonate - **Uniform Reliable Group Communication Protocol** (URGC)
– Transmitere multicast de articole noi pe Mbone – **Muse**
– Transmitere de fisiere - **Multicast File Transfer Protocol** (MFTP)
– Evidenta pachetelor transmise intr-un server de logging – **Log-Based Receiver-reliable Multicast** (LBRM)

## Multicast IP

Transmiterea multicast se intalneste si la nivelul IP al ierarhiei de protocoale TCP/IP. Ea se bazeaza pe cateva ingrediente:
- folosirea adreselor IP de clasa D pentru transmitere pachete (vezi figura in continuare)
- utilizarea protocolului Internet Group Management Protocol (IGMP) pentru gestiunea grupurilor
- folosirea ruterelor multicast care:
    transmit pachete multicast membrilor (best effort)

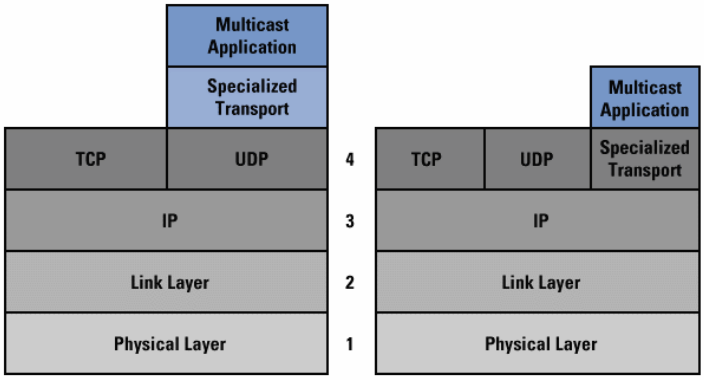
verifica periodic host-urile ramase in grup (pool – facilitate prezeanta in IGMP1)
sunt notificate la intrarea unui nou host in grup.

Problema este ca muticast IP nu este fiabil, ca de altfel nici unicast IP.




## Transport - Multicast peste UDP sau IP

Pentru unicast fiabilitatea este realizata de TCP, care face corectia erorilor la transmitator. Extinderea solutiei TCP la transmisia multicast nu merge, deoarece ea ar presupune:
- receptia multor mesaje de confirmare ACK sau NAK la transmitator, care ar reprezenta un pericol de gatuire a sistemului
- evidenta greoaie a setului de receptori de catre transmitator (cine a primit corect, cui trebuie sa retransmita etc)
- conditii diferite pentru receptori diferiti (round-trip time, delay*bandwidth, legaturi supraincarcate diferite), care ingreuneaza stabilirea unor parametri importanti ai transmitatorului (de ex. stabilirea timeout-ului, a ferestrei de congestie etc.)

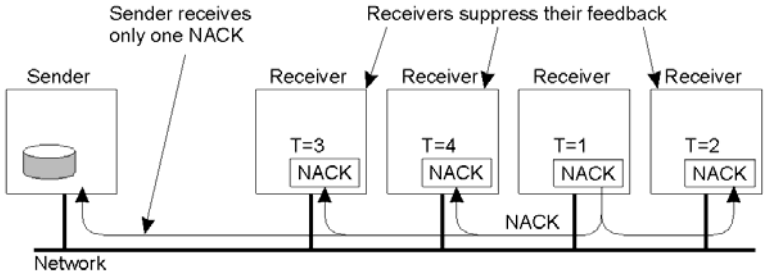



Alta solutie este un Transport specializat peste UDP sau IP (vezi figura) care sa evite controlul transferurilor la transmitator.

## Scalable Reliable Multicast - SRM

SRM este bazat pe IP multicast, care este ne-fiabil, dar care ofera cateva avantaje importante:
– Sursa transmite pe o adresa multicast fara sa cunoasca membrii grupului
– Un receptor intra in- sau iese din grup fara a afecta ceilalti membri.
La acestea, SRM adauga caracteristici similare TCP si anume:
– Fiabilitate capat la capat

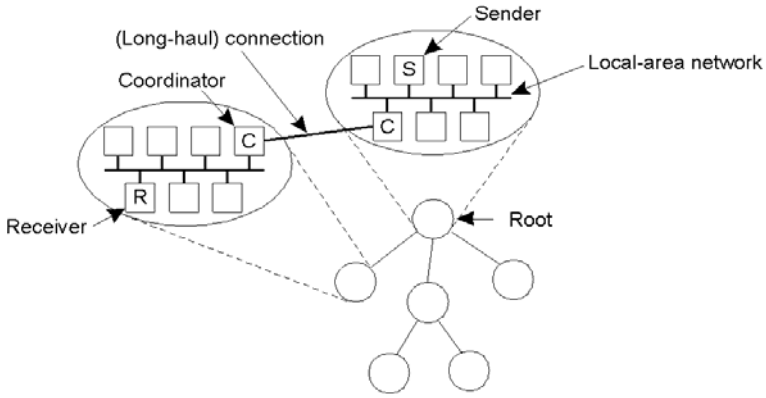
– Adaptare la conditiile retelei (dimensiune, trafic, topologie).
Ideea SRM este urmatoarea: fiecare membru al grupului raspunde de receptia corecta a mesajelor care ii sunt adresate (corectia se face la receptor).




Fiecare mesaj poarta un numar de secventa, iar mesajele sunt primite in ordinea in care sunt transmise. Receptorii pot folosi numerele de secventa pentru a pastra o evidenta a mesajelor primite. Periodic, fiecare receptor transmite mesaje *heartbeat* cu numarul de secventa al ultimului pachet primit. Ceilalti receptori pot astfel detecta daca le lipseste vreun pachet. Daca un receptor are un pachet lipsa, el trimite un mesaj *NACK* cerand retransmiterea pachetului. Transmitatorul retransmite un mesaj *Repair* cu pachetul respectiv, pe care il ia din cache.

**Optimizari**
Optimizarile se bazeaza pe faptul ca mesajele sunt transmise de receptori in regim broadcast astfel ca acestea pot fi vazute de ceilalti membri ai grupului.
Pentru a minimiza numarul de *NAK* si *Repair*, receptorul asteapta un timp aleator si, daca un alt receptor a trimis un mesaj NAK atunci nu il mai trimite pe al sau. Altfel face transmiterea acestuia. Problema este planificarea intarzierilor pentru a asigura reducerea numarului de mesaje la un singur NAK.
Pentru evitarea consumului de resurse pentru cei care au primit corect pachetele, retransmiterea se poate face punct la punct sau multicast doar pentru gruparea de procese care au pierdut acelasi mesaj. In fine, pentru accelerarea procesului de recuperare, retransmiterea pachetului poate fi facuta de un membru "local" al grupului, care l-a primit corect.

Imbunatatiri ale protocolului pot fi obtinute prin varianta de control ierarhic, ilustrata in figura urmatoare.

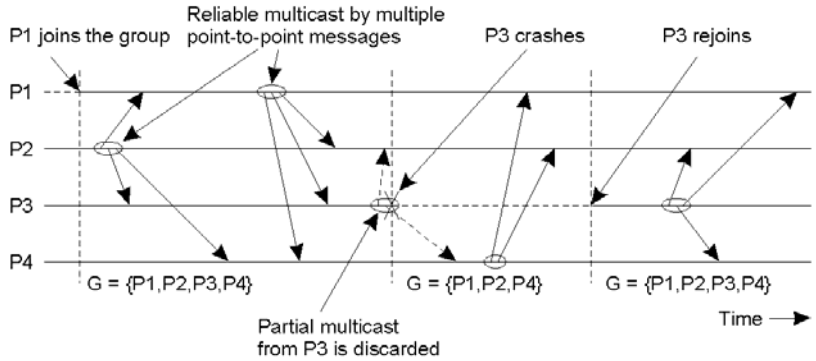


Grupul este impartit in subgrupuri cu **coordonatori locali** care retransmit mesajele catre fii. In plus, coordonatorii fac agregarea cererilor de retransmitere de la fii asigurand reducerea numarului cererilor care ajung la transmitator. In fine, un coordonator local are propriul sau buffer in care pastreaza mesajele receptionate, pe care le poate furniza fiilor. Problema este constructia dinamica a arborelui, pentru care se pot utiliza arbori multicast identificati de reteaua suport sau solutii la nivelul aplicatiei (overlay network).

## Multicast Atomic

Multicastul atomic este un multicast fiabil chiar daca unele procese se defecteaza. Procesul defect este scos din grup; la refacere el se va alatura din nou grupului, ceeace inseamna refacerea starii sale in acord cu starea proceselor din grup. Asta inseamna ca trebuie sa se cunoasca exact ce operatii lipsesc si in ce ordine trebuie executate. O problema este cea in care transmitatorul se defecteaza in timpul transmiterii unor mesaje.

Regula este ca nu se accepta ca un mesaj sa fie livrat unor procese din grup si nelivrat altora. Deci, un mesaj este:
– fie livrat tuturor proceselor din grup
– fie nelivrat.
Regula este acceptabila cand transmitatorul cade in timpul transmiterii, situatie similara cu caderea transmitatorului inaintea transmiterii mesajului. In plus, mesajele sunt livrate in aceeasi ordine tuturor proceselor.
Protocolul multicast atomic este bazat pe notiunea de **group view**, care reprezinta setul de procese din grup "vazut" de transmitator cand a trimis mesajul si carora trebuie sa li se livreze mesajul.

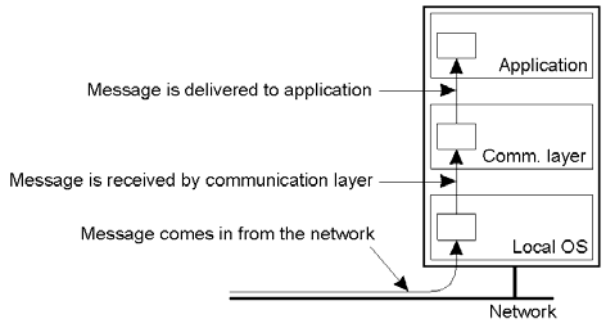



Toate procesele din set au acelasi view dar un group view se poate schimba datorita caderii unor procese. Toate operatiile multicast **au loc intre schimbarile de view**.
Sa consideram un exemplu! Fie o colectie de patru procese (vezi figura anterioara). La un anumit moment, P1 se alatura grupului si group view devine G={P1, P2, P3, P4}. P2 transmite multicast (prin mai multe mesaje punct la punct) un mesaj si acesta este receptionat de celelalte trei procese din group view. Ulterior, P1 transmite un mesaj grupului. Apoi, procesul P3 incepe sa transmita un mesaj dar se defecteaza pe parcursul transmiterii. Ca urmare, mesajul transmis de el nu este livrat si group view devine G={P1,P2,P4}. Cand P3 revine in grup se reia transmiterea nereusita.
Obs. Receptia unui mesaj este diferita de livrarea lui catre aplicatie. Detalii in partea de implementare prezentata in continuare.

Anuntul de schimbare a unui group view se face prin transmiterea multicast a unui mesaj special **vc** – view change. Cu aceasta, principiul **multicast-ului sincron virtual** se poate formula prin raportare la view change, astfel:
Daca **m** este transmis proceselor din G si, inainte de terminare, se transmite **vc** atunci toate procesele din G primesc **m** inainte de **vc**, sau nici un proces nu primeste **m**.
Mesajul m transmis lui G nu poate fi livrat decat proceselor din G.

## Implementare Virtual Synchrony (1)

Pentru ilustrarea implementarii sincronismului virtual adoptam urmatoarele ipoteze:
– se utilizeaza comunicarea fiabila punct-la-punct (TCP)
– se face transmitere multicast – trimitere mesaj **m** fiecarui membru din grup
– mesajele de la aceeasi sursa sunt primite de nivelul de comunicare in ordinea trimiterii
– arhitectura sistemului este cea din figura (in care se arata diferenta intre receptia unui mesaj si livrarea mesajului)



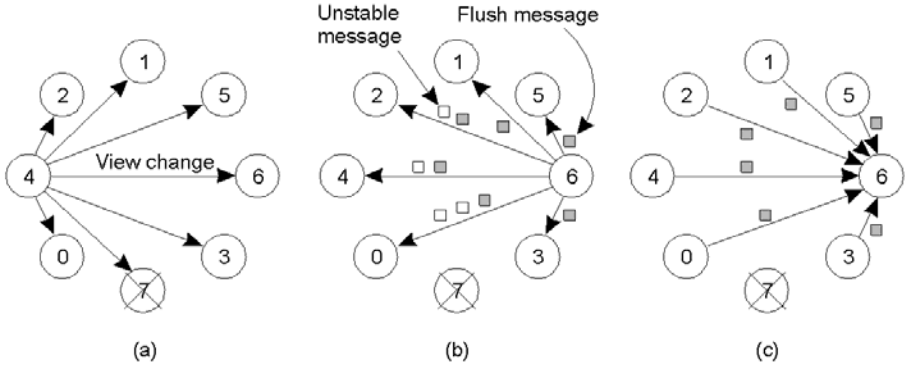

## Implementare Virtual Synchrony (2)

Cerinta este ca un mesaj trimis unui group view G sa fie livrat tuturor proceselor din G inainte de urmatoarea schimbare de view.
Probleme pot apare daca **vc** apare inainte ca mesajul m sa fie livrat tuturor proceselor din G. In plus, daca transmitatorul cade, exista o problema suplimentara si anume modalitatea in care iau mesajul procesele care inca nu l-au primit de la procesul cazut.
Solutia se bazeaza pe diferentierea intre doua categorii de mesaje:
– mesaje **stabile** (cele primite de toate procesele); aceste mesaje sunt livrate;
– mesaje **instabile**; ele sunt pastrate de procesele care le-au primit, astfel ca unul din ele il va trimite tuturor celorlalte procese.

## Implementare Virtual Synchrony (3)

In exemplul din figura urmatoare, procesul 4 observa ca 7 a cazut si trimite un mesaj view change. Cand procesul 6 primeste view change, el trimite toate mesajele instabile (pe care le schimba totodata in stabile), urmate de un mesaj *flush* (care marcheaza ca nu mai are mesaje instabile de trimis). Asta garanteaza ca fiecare mesaj care a fost primit de cel putin un proces din Gi este primit de fiecare proces care era in $G_i$ si a ramas in urmatorul group view care nu mai include procesul cazut, $G_{i+1}$. Cand un proces Q primeste mesajele, el ignora duplicatele (mesajele pe care le-a primit deja) si accepta mesajele neprimite anterior, livrand toate mesajele in ordinea

corecta. Procesul 6 instaleaza noul group view cand primieste un *flush* de la toti ceilalti (fiecare proces valid procedeaza la fel ca 6 la primirea mesajului view change).




## Generalizare – distributed commit

Distributed commit asigura ca, data fiind o operatie distribuita proceselor unui grup:

sau fiecare proces din grup executa operatia

sau nici un proces nu o executa.

Solutiile cunoscute folosesc un coordonator. Sunt trei variante majore:

one-phase commit – nu poate trata caderea unui proces

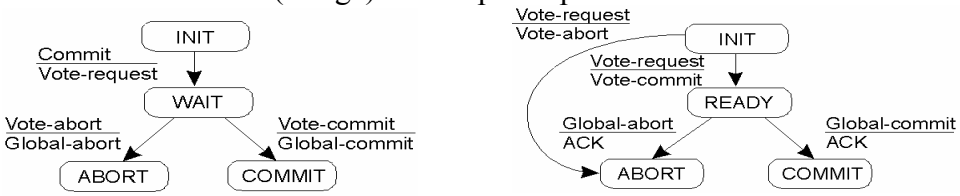
two-phase commit – nu poate trata caderea coordonatorului

three-phase commit.

Ne referim in continuare la ultimele doua modele.

## Two-Phase Commit (2PC)

Solutiile folosesc un Coordonator, care are rolul de a initia operatia. Urmatoarele diagrame de stari corespund coordonatorului (stanga) si unui participant.




Operatiile sunt usor de urmarit, diagramele prezentand explicit starile si tranzitiile intre stari. Fiecare tranzitie este etichetata cu perechea intrare/iesire asociata tranzitiei. La comitere, coordonatorul trimite un mesaj *Vote-request* tuturor participantilor si asteapta raspunsurile lor. Coordonatorul colecteaza raspunsurile si decide sa aborteze daca primeste cel putin un mesaj *Vote-abort* sau sa comita daca toate mesajele primite sunt *Vote-commit*.

Cand un participant primeste un mesaj *Vote-request* el raspunde cu *Vote-abort* sau cu *Vote-commit*. Apoi asteapta decizia coordonatorului. Daca decizia este *Global-abort* atunci aborteaza, alfel comite.

Dupa cum se vede, exista stari in care coordonatorul (starea WAIT) sau un participant (starile INIT, READY) asteapta primirea unui mesaj. Pentru evitarea blocarilor se folosesc timeout-uri. Protocolul specifica ce se intampla la fiecare timeout. Avem urmatoarele situatii.

Comportarea participantului la timeout in INIT – decide sa aborteze local tranzactia si trimite VOTE_ABORT coordomatorului.

Comportarea coordonatorului la timeout in WAIT – decide sa aborteze si trimite GLOBAL_ABORT participantilor.
Comportarea participantului la timeout in READY – participantul asteapta decizia finala a coordonatorului, care ar putea fi indisponibila pentru o perioada de timp sau poate fi pierduta. Deoarece participantul nu poate lua decizii locale, solutia este de a lasa participantul sa contacteze un alt participant *Q* si ia deciziii in functie de starea acestuia. Rezumarea actiunilor este inclusa in urmatorul tabel impreuna cu justificarile fiecarei decizii. Asa cum se vede, decizia este in general posibila, cu exceptia cazului in care toti participantii sunt in READY si trebuie sa astepte recuperarea coodonatorului.

| **Starea lui Q** | **Actiunile lui P** | **Justificare** |
|---|---|---|
| COMMIT | Trece in COMMIT | P a pierdut GLOBAL_COMMIT |
| ABORT | Trece in ABORT | P a pierdut GLOBAL_ABORT |
| INIT | Trece in ABORT | Coord a crapat inainte de trimitere VOTE_REQUEST tuturor participantilor |
| READY | Contacteaza alt participant | Daca toti READY atunci asteapta recuperare coordonator |

**Implementare: logare stare ptr refacere la recuperare**

Pentru recuperarea proceselor din defecte este necesara logarea starii in memoria persistenta. Dupa recuperarea din defect, procesul va relua executia de la ultima stare logata. Procedurile urmatoare descriu functionarea coordonatorului si a participantilor, inclusiv logarile realizate.

**Log la Coordonator**

Coordonatorul trebuie sa inregistreze doua situatii:
- START_2PC – marcheaza intrarea in starea WAIT; la repornire dupa cadere, coord va relua de la aceasta stare si va retransmite VOTE_REQUEST tuturor participantilor.
- GLOBAL_ABORT sau GLOBAL_COMMIT – similar, coordonatorul retransmite decizia sa catre participanti.

```
write START_2PC to local log;
multicast VOTE_REQUEST to all participants;
while not all votes have been collected {
   wait for any incoming vote;
   if timeout {
      write GLOBAL_ABORT to local log;
      multicast  GLOBAL_ABORT to all participants;
      exit;}
   record vote;                  /*inregistreaza un vot*/
}
if all participants sent VOTE_COMMIT and coordinator votes COMMIT{
   write GLOBAL_COMMIT to local log;
   multicast GLOBAL_COMMIT to all participants;}
```

```
else {
   write GLOBAL_ABORT  to local log;
   multicast GLOBAL_ABORT to all participants;
}
```

**Logare stare la participant**

La recuperarea participantului dupa cadere, in functie de log, se fac urmatoarele:
- INIT – participantul aborteaza, la fel ca la timeout.
- GLOBAL_COMMIT sau GLOBAL_ABORT – participantul reia din starea respectiva si retransmite decizia la coordonator.
- VOTE_COMMIT – la fel ca la timeout.

```
write INIT to local log;
wait for VOTE_REQUEST from coordinator;
if timeout {
   write VOTE_ABORT to local log;
   exit;
}
if participant votes COMMIT {
   write VOTE_COMMIT to local log;
   send VOTE_COMMIT to coordinator;
   wait for DECISION from coordinator;
   if timeout {
      multicast DECISION_REQUEST to other participants;
      wait until DECISION is received; /* remain blocked */
      write DECISION to local log;
   }
   if DECISION == GLOBAL_COMMIT
      write GLOBAL_COMMIT to local log;
   else if DECISION == GLOBAL_ABORT
      write GLOBAL_ABORT to local log;
} else {
   write VOTE_ABORT to local log;
   send  VOTE_ABORT to coordinator;
}
```

Operatiile unui participant pentru tratarea cererilor de decizie de la alti participanti.

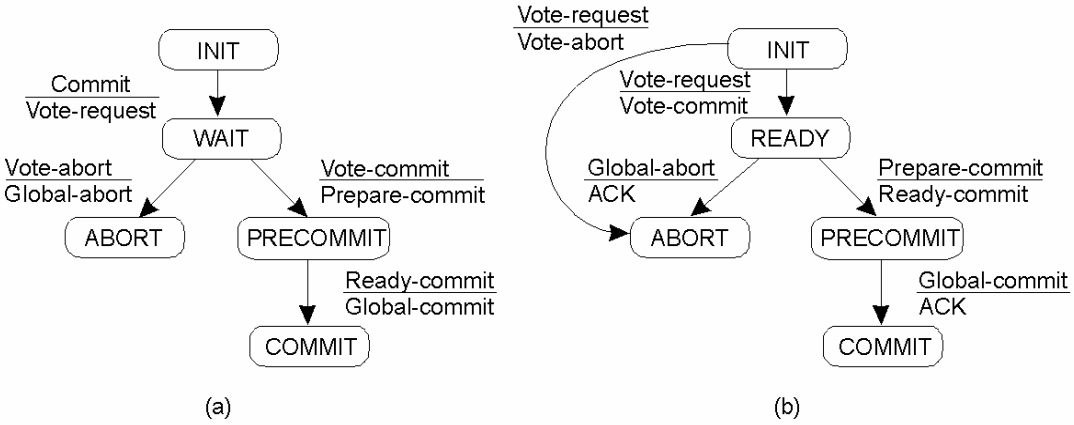
```
actions for handling decision requests: /* executed by separate thread */
while true {
   wait until any incoming DECISION_REQUEST is received; /* remain blocked */
   read most recently recorded STATE from the local log;
   if STATE == GLOBAL_COMMIT
      send GLOBAL_COMMIT to requesting participant;
   else if STATE == INIT or STATE == GLOBAL_ABORT
      send GLOBAL_ABORT to requesting participant;
```

```
   else
      skip;  /* participant remains blocked */
}
```

La o analiza simpla, se poate vedea ca participantii nu pot decide atunci cand toti au primit si procesat VOTE_REQUEST, deci starea tuturor este READY. In aceasta situatie se executa **skip** in algoritmul precedent. Ca urmare, participantul care a trimis DECISION_REQUEST nu primeste nici un raspuns si ramane blocat definitiv.

## Three-Phase Commit (3PC)

Protocolul three-phase commit rezolva neajunsul blocarii participantilor la caderea coordonatorului. In figura urmatoare se prezinta masina de stari pentru coordonator in 3PC si masina de stari pentru un participant.
Actiunile sunt urmatoarele:
1a. Coordonatorul trimite *Vote-request* participantilor si asteapta
1b. Cand un participant primeste *Vote-request*, el raspunde *Vote-commit* sau *Vote-abort*; la *Vote-abort* abandoneaza operatia locala
2a. Coordonatorul colecteaza voturile de la participanti. Cand toate sunt *Vote-commit* trimite *Prepare-commit* tuturor, altfel trimite *Global-abort* si se opreste
2b. Participantul asteapta decizia. Daca este *Global-abort* atunci confirma si se opreste. Daca este *Prepare-commit*, transmite *Ready-commit* si asteapta
3a. Coordonatorul asteapta *Ready-commit* de la toti si transmite *Global-commit*
3b. Participantul asteapta *Global-commit* si confirma




La fel ca in cazul anterior analizam actiunile la **timeout**. Problema este daca un participant poate decide ce trebuie sa faca dupa o cadere si recuparare in READY sau PRECOMMIT, chiar daca alti participanti sau coordonatorul au cazut. **Principiul** folosit de protocol face ca, pe calea spre COMMIT, coordonatorul si participantii sa nu difere prin mai mult de o tranzitie. Ca urmare, un participant, la un timeout in starea READY poate afla de la coordonator sau de la alti participanti daca trebuie sa aborteze sau sa treaca in PRECOMMIT.
Avem urmatoarele situatii.
Coordonator in WAIT –> *Global-abort* ( presupune ca un participant a cazut)

Coordonator in PRECOMMIT –> *Global-commit* (presupune ca un participant a cazut dar el a votat pentru comitere si va recupera)
Participant in INIT –> *Vote-abort*.

**Participant P in READY sau PRECOMMIT** -> contacteaza alti participanti Q (coordonatorul a cazut). Daca:
un Q in COMMIT sau ABORT -> P trece in starea respectiva
un Q in INIT (obligatoriu coord si nici un participant nu sunt in PRECOMMIT) -> P aborteaza
toti in PRECOMMIT -> P trece in COMMIT (toti au trimis Ready-commit si coord a cazut; cand va recupera va trece in COMMIT).

**Daca doar unii participanti pot fi contactati**
*toti ce pot fi contactati* sunt in PRECOMMIT -> P trece in COMMIT (un proces cazut va recupera in READY, PRECOMMIT sau COMMIT – oricum, va ajunge in final la COMMIT)
*toti ce pot fi contactati* sunt in READY -> P aborteaza (un participant a cazut si va recupera in INIT, ABORT sau PRECOMMIT; in toate cazurile el va ajunge sa aborteze).

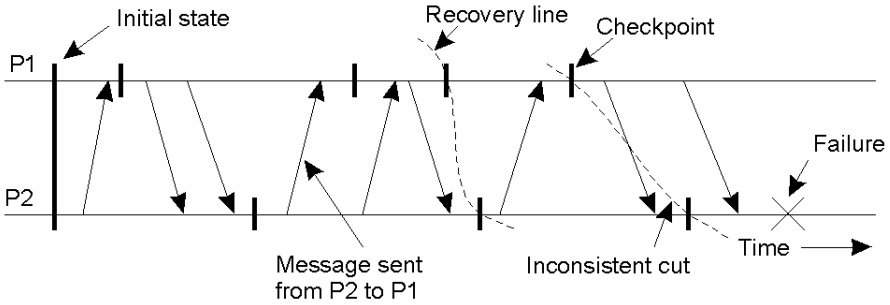
**Concluzie**: procesele supravietuitoare pot lua intotdeauna o decizie.

## Recuperarea

La o cadere, sistemul trebuie adus intr-o stare corecta. Sunt doua solutii:
- sistemul este adus la o stare anterioara (backward recovery)
- sistemul este trecut intr-o stare noua (forward recovery).
Recuperare inapoi este mai folosita, dar mai complicata in sistemele distribuite (deoarece procesele trebuie sa identifice o stare consistenta). Sunt folosite doua tehnici de recuperare:
– checkpointing (inregistrarea starii proceselor);
– message logging (logarea mesajelor, folosita in combinatie cu checkpointing).

## Checkpointing

Starea globala consistenta este un instantaneu distribuit constituit din starile proceselor, salvate in **memorii stabile** locale. Memoriile stabile rezista caderilor proceselor astfel ca checkpointurile pot fi regasite in aceste memorii, la recuperare.

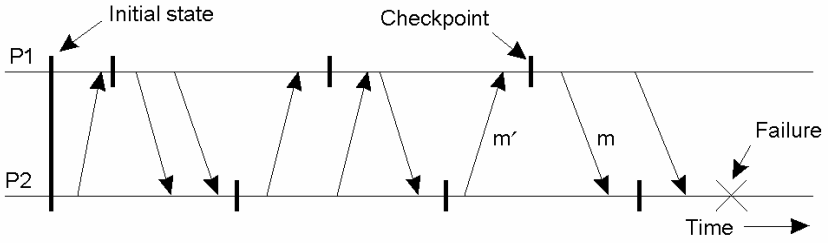



Consistenta se bazeaza pe o regula simpla: daca P1 a inregistrat o receptie de mesaj atunci starea trebuie sa includa o transmitere la un alt proces P2. In figura anterioara sunt ilustrate doua

procese cu diferite checkpointuri si doua taieturi: una este inconsistenta, cealalta este consistenta si poate constitui o linie de recuperare in cazul unei defctari.
Linia de recuperare este, de fapt, cel mai recent instantaneu distribuit (cea mai recenta taietura consistenta).

### Checkpointing Independent

Linia de recuperare este greu de gasit iar procesul sufera potential de efectul de domino. In figura urmatoare, P2 crapa si se intoarce la checkpoint. P1 face la fel, dar checkpointurile celor doua procese nu se potrivesc. Ca urmare, P2 da inapoi la precedentul checkpoint, dar si aceasta taietura este inconsistenta. Ca urmare, P1 da si el inapoi si asa mai departe. In final, se ajunge la starea intitala.




### Implementare

In esenta, fiecare proces ia (independent de alte procese) checkpointuri. Pentru recuperare este nevoie de inregistrarea dependentelor intre checkpointuri luate de diferite procese. Folosim urmatoarele notatii:
*CP[i](m)* denota checkpoint *m* al procesului *Pi*
*INT[i](m)* interval intre *CP[i](m-1) si CP[i](m)*.
Ideea este de a inregistra, la CP-uri, dependenta intervalelor. Protocolul este urmatorul:

*Pi* trimite mesaj in *INT[i](m)* si adauga (i,m) ptr receptor
*Pj* primeste mesaj in *INT[j](n)*
inregistreaza dependenta *INT[i](m) -> INT[j](n)*
la checkpoint CP[j](n): salveaza (*INT[i](m) -> INT[j](n)*) in memoria stabila.

Daca *Pi* revine la *CP[i](m-1)* atunci *Pj* revine la CP[j](n-1) sau la un CP anterior.
OBS: Daca starile nu sunt consistente sunt necesare alte reveniri.

### Checkpointing Coordonat

O **solutie** simpla este protocolul cu blocare, in doua faze. Ideea lui este ca fiecare proces face un checkpoint dupa o actiune globala coordonata. Protocolul are urmatoarea descriere:

Coordonator: trimite multicast cerere *checkpoint*
Participant: primeste o cerere *checkpoint*
face checkpoint
opreste transmiterea mesajelor (le pune intr-o coada) care ii sunt pasate de aplicatia pe care o executa
raporteaza ca a facut checkpoint
Coordonator: asteapta toate confirmarile de checkpoint

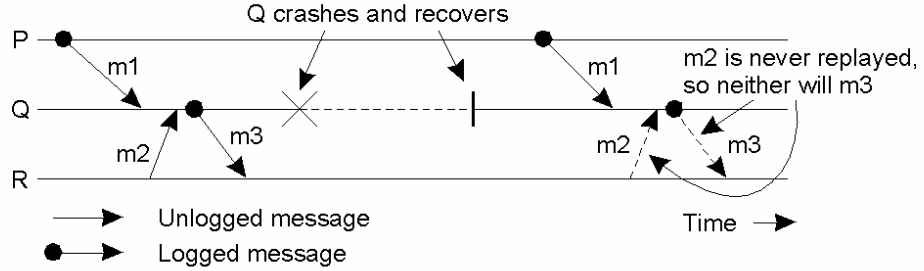
difuzeaza *checkpoint done*
Participant: primeste *checkpoint done*
continua

Acest protocol are dezavantajul ca permite inconsistenta: in CP nu se inregistreaza receptiile mesajele transmise de participanti si aflate inca in tranzit.

## Logarea Mesajelor

Starea consistenta globala poate fi atinsa si prin "repetarea" mesajelor transmise anterior aparitiei defectului. Ideea se bazeaza pe un model de executie determinist pe bucati, in care executia unui proces (pe care o putem reprezenta ca o succesiune de stari) este vazuta ca o succesiune de intervale de stari. Fiecare interval incepe cu un eveniment nedeterminist, de exemplu receptia unui mesaj, dar in interval executia este determinista. Aceasta permite ca, in vederea recuperarii, sa se inregistreze doar evenimentele nedeterministe.

Una din problemele "cheie" se refera la "Cand trebuie inregistrate mesajele?" Inregistrarea trebuie facuta astfel incat sa se evite aparitia proceselor **orfane**.
Un proces R este orfan daca rezista caderii unui alt proces Q dar a carui stare este inconsistenta cu starea lui Q dupa recuperare.
**Exemplu**. Q a primit de la P si a livrat (aplicatiei) mesajul m1, apoi logheaza m1. Q primeste apoi m2, incepe logarea lui m2, dar pentru a mari eficienta sistemului (logarea este o operatie costisitoare) nu asteapta terminarea logarii lui m2 si transmite mesajul m3 lui R. R primeste m3 si il livreaza. Q cade inainte de terminarea operatiei de logare astfel ca logarea lui m2 nu se face. La reluarea lui Q dupa defectare, Q "rejoaca" mesajul m1 (logat) si executa operatiile deterministe declansate de acest eveniment. Dar, m2 (nelogat) nu este rejucat si, foarte probabil, Q se executa pe o cale in care nu se mai transmite m3. Ca urmare, R a livrat un mesaj (m3) venit de la Q (inainte de defectare) dar a carui transmisie nu are loc (la rejucarea mesajelor). Deci, R devine orfan, starea sa fiind inconsistenta cu cea a lui Q dupa recuperare.




## Scheme de logging

Schemele de logare inregistreaza urmatoarele informatii despre mesaje.

**HDR[m]**: antetul mesajului contine toate informatiile necesare retransmiterii mesajului si livrarii sale in ordinea corecta: sursa, destinatia, numarul de secventa, numarul de livrare.

Mesajul *m* este **stabil** cand nu mai poate fi pierdut (a fost pus in memoria stabila). Mesajele stabile pot fi "rejucate" pentru recuperare.

**DEP[m]**: include procesele la care s-a livrat *m* si procesele la care s-a livrat *m'* dependent cauzal de livrarea lui *m*.

**COPY[m]**: contine procesele care au o copie a lui *m* in memoria volatila (inca nu in memoria stabila locala), copie care ar putea fi utilizata pentru rejucarea lui *m*. Daca toate procesele din COPY[m] cad atunci *m* nu mai poate fi rejucat.

***C*** este colectia proceselor cazute.

Cu aceste notatii, *Q* este **orfan** daca
- depinde de un mesaj **m** (Q este in DEP[m]) si
- **m** nu poate fi retransmis deoarece procesele care au o copie a lui *m* au cazut, adica
COPY[*m*] este inclus in C.

## Scheme de logging (2)

**Scopul** schemelor de logare este evitarea orfanilor. Aceasta evitare este posibila daca, atunci cand procesele din COPY(m) au cazut, nu ramane nici un proces in DEP(m). Conditia poate fi asigurata daca orice proces dependent de *m* pastreaza o copie al lui *m*, adica:
pentru fiecare mesaj *m*, DEP[m] este inclus in COPY[m]

**Solutii**
Discutam doua variante, una pesimista si una optimista.
**Protocolul pesimist** porneste de la premisa ca defectarile sunt frecvente. El cere ca, daca P primeste un mesaj *m*, P sa il faca stabil (sa il scrie in memoria stabila) inainte de a trimite mesajul urmator. Cu alte cuvinte, pentru fiecare mesaj *nonstabil m*, exista cel mult un proces dependent de *m*, adica |DEP[m]|<=1

**Protocolul optimist** presupune ca defectarile sunt improbabile si actioneaza dupa o "cadere." Fiecare proces orfan Q este intors la o stare in care Q nu este in DEP[m]. Cu alte cuvinte, pentru fiecare mesaj instabil *m*, daca COPY[m] este inclus in C, atunci DEP[m] este inclus de asemena in C.